BDD is a great way for teams to have a conversation about which nonfunctional requirements really matter for a particular application. BDD helps phrase these nonfunctional requirements in terms that stakeholders can understand and help define, and that can be easily verified and potentially automated.

In addition, not all nonfunctional requirements are equal. Expressing requirements as BDD scenarios can help stakeholders and developers determine what areas are most affected by nonfunctional requirements, such as performance or security.

For example, before doing any performance testing on an application, a team should ask end users and managers about their idea of acceptance performance metrics and the application's anticipated load. Ideally, these expectations should be expressed in a way that's meaningful to the business, but this isn't always the case in practice. Many teams spend time and effort measuring performance metrics such as CPU and memory usage that aren't necessarily related to the underlying business goals and don't provide useful feedback to the business.

Let's look at how this might work in practice. The Flying High Flight Status application relies on receiving timely updates from the flight status message server. One of the selling points of the Frequent Flyer program is its ability to provide clients with fast updates about flight status, including information about delays, expected arrival times, gates, baggage carousel numbers, and so forth.

From discussion with stakeholders and the marketing department, which had conducted market research and studied competing products, it emerged that updates needed to be dispatched within 30 seconds 95% of the time, and within 60 seconds 100% of the time, and that every update needed to be successfully dispatched.

You could express this performance requirement like this:

```
Feature: Application performance

  Scenario: Flight status update performance
    Given the flight status server is running
    When a production peak hour of updates is sent at 5 times production
      throughput
    Then 95% of the updates should be received within 30s
    And 100% of the updates should be received within 60s
    And 100% of the updates should be received successfully
```

Once you've expressed the performance requirements in these terms, you can automate. There are many open source and commercial load-testing tools, and most can be scripted. Popular open source options in the Java world include SoapUI (www.soapui.org), JMeter (http://jmeter.apache.org/), and The Grinder (http://grinder.sourceforge.net). These tools let you define and execute test scripts that simulate interactions with an application, such as requesting a web page, invoking a web service, and querying an EJB. To simulate load, they can run scripts on a number of remote machines as well as locally. They're also relatively easy to automate and to run programmatically. This makes them easy to integrate into tools such as Cucumber and JBehave.

Load testing like this needs realistic production-like data. In general, the best way to get an idea of the number of messages sent during peak times would be to observe